DIA 1 | Leitura e listagem de JSON

Trilha autônoma | Programação Back-End | Node.js + fs

Situação de aprendizagem

Você faz parte de uma equipe de desenvolvimento de uma fábrica. Na aula anterior, utilizou Node.js e o módulo fs para gravar registros em JSON (CREATE). Agora deverá recuperar e consultar esses registros (READ), sem alterar o arquivo.

Duração: 5 aulas de 45 minutos (225 minutos). Trabalho individual. O professor presente acompanha a organização e não precisa programar por você.

Objetivos

Ler um arquivo JSON, converter texto em objetos JavaScript, percorrer um array, implementar uma função de listagem e explicar as decisões adotadas.

Prepare o ambiente

Extraia a pasta Projeto_Aluno, abra no VS Code e verifique no terminal node -v. Examine maquinas.json. Não modifique os valores: eles serão usados nos testes automáticos.

Aula 1 | 45 min — Revisão e diagnóstico

10 min: recordar CREATE e READ; 15 min: observar o arquivo maquinas.json; 15 min: responder às perguntas; 5 min: registrar dúvidas.

Questões: (1) O que significa persistência? (2) Qual a diferença entre gravar e ler um arquivo? (3) O JSON é um objeto JavaScript ou um texto no arquivo? (4) Quantos registros existem? (5) Qual é a utilidade de um ID único?

Aula 2 | 45 min — Ler texto com fs

15 min de leitura orientada; 20 min experimentação; 10 min registro. O método fs.readFileSync(caminho, "utf8") lê o conteúdo textual. A opção "utf8" indica a codificação usada na leitura. A leitura não modifica o arquivo.

```
const fs = require('node:fs');
const texto = fs.readFileSync('maquinas.json', 'utf8');
console.log(typeof texto); // string
console.log(texto.slice(0, 100));
```

Exercício: explique por que uma string, sozinha, não permite acessar diretamente cada máquina por nome ou status. Teste o comportamento com o arquivo de exemplo.

Aula 3 | 45 min — Interpretar JSON

15 min: ler explicação; 20 min: codificar; 10 min: responder. JSON.parse(texto) interpreta texto JSON válido e o converte em um valor JavaScript. Neste projeto, esse valor é um array de objetos.

```
const maquinas = JSON.parse(texto);
console.log(Array.isArray(maquinas)); // true
console.log(maquinas.length);        // 8
console.log(maquinas[0].nome);       // Torno CNC T-01
```

Exercício: identifique no arquivo quais são as propriedades de cada registro, os tipos dos valores e o motivo de utilizar JSON.parse().

Aula 4 | 45 min — Percorrer um array

10 min de revisão; 25 min prática; 10 min discussão escrita. forEach() executa uma função para cada item do array; use-o para imprimir id, nome e status.

```
maquinas.forEach((maquina) => {
  console.log(maquina.id, maquina.nome, maquina.status);
});
```

Exercício: implemente listarMaquinas(maquinas) no arquivo app.js. Acrescente um separador visual entre os registros e observe a ordem de exibição.

Aula 5 | 45 min — Consolidar e entregar

25 min: completar carregarMaquinas() e listarMaquinas(); 10 min: executar node app.js ou npm start; 10 min: produzir relatório parcial. Não é necessário resolver find e filter hoje.

Evidências do Dia 1

Entregar: app.js parcial, captura de tela da listagem com 8 registros, e relatório em PDF com respostas às perguntas e explicação de cada linha usada para leitura/conversão. Nome sugerido: Turma_Nome_Dia1_READ.pdf.

Checklist

[ ] Li o JSON [ ] Confirmei 8 máquinas [ ] Converti com JSON.parse [ ] Exibi a listagem [ ] Registrei minha explicação [ ] Enviei pelo Classroom.

Apoio do NotebookLM

Use a fonte disponibilizada pelo professor. Pergunte: “Explique a diferença entre ler um arquivo e interpretar um JSON, com um exemplo.” Registre, com suas próprias palavras, uma conclusão. Não aceite códigos sem compreender como executá-los.

Se algo não funcionar

Confirme se está na pasta certa, se o nome maquinas.json corresponde ao arquivo, se executou Node.js no terminal e se não há erros de sintaxe. Se o erro persistir, registre a mensagem e o procedimento realizado no relatório.